ChatGPT

Modern AI Models Field Guide for April 2026

Executive summary

Scope and date. This guide maps the practical AI model landscape **as of April 6, 2026** (Europe/Kyiv), focusing on how to *read model names*, separate *architecture vs training vs packaging*, and make sense of the ecosystem without vendor hype. It is **not** a stack recommendation and **not** a benchmark-only comparison. Many examples use official model strings and commonly encountered open-weight artifact names. [1]

A high-value mental map of "the model world" today

Most confusion comes from mixing three different layers:

Layer one: the model's job (function).
General chat LLMs, coding LLMs, embeddings, rerankers, multimodal/VLMs, ASR/TTS, image/video generators.

Layer two: the model's nature (architecture + training).
Dense vs MoE, decoder-only vs encoder-only vs seq2seq, diffusion vs transformer-based diffusion, "reasoning tuning", tool-use training, long-context training.

Layer three: the artifact you actually run (packaging).
Original checkpoints (often safetensors), quantized variants (AWQ/GPTQ/GGUF/EXL2), engine-specific builds (TensorRT-LLM, ONNX), plus runtime constraints (vLLM vs llama.cpp vs MLX).

If you keep those layers separate, model "names" become far less intimidating.

Major categories of AI models, in practical terms

The list below is intentionally "applied": it's about what you buy/run/call.

Category	What it does	Typical input → output	Where you see it	Common evaluation pitfalls
General-purpose LLMs	Broad language tasks, chat, assistants	text → text (sometimes image/audio too)	API model IDs (e.g., GPT / Claude / Gemini families), open-weight "Instruct" models	Vendor benchmark cherry-picks; prompts differ; tool access changes outcomes [2]

Category	What it does	Typical input → output	Where you see it	Common evaluation pitfalls
Small language models (SLMs)	LLM-like behavior under tight latency/VRAM	text → text	“mini/nano/small” lines; 0.5B–~10B class open weights	“Small but tuned” can beat older bigger baselines; comparisons often unfair [3]
Reasoning-tuned models	Better multi-step reliability by training + test-time compute	text → text (sometimes with hidden reasoning tokens)	“reasoning,” “thinking,” “effort,” “verifier,” “trace” features	Longer answers ≠ better reasoning; hidden reasoning tokens complicate cost comparisons [4]
Coding models	Code completion, repo editing, agentic SWE tasks	text/code → code/text	“coder,” “codestral,” “deepseek-coder,” “devstral”	“HumanEval wins” ≠ “fixes your codebase”; tool-use matters [5]
Embedding models	Turn text (or multimodal inputs) into vectors for search/RAG	text → vector	“embedding,” “embed,” “text-embedding-*”	Dimensionality and pooling choices affect results; MTEB ≠ your domain [6]
Rerankers	Re-score candidate docs for a query (often cross-encoders)	(query, docs) → scores	“rerank,” “ranker” endpoints	People skip reranking and blame embeddings; latency trade-off [7]
Multimodal models	Handle multiple modalities (text+image+audio+video)	multi → text (and sometimes audio/image outputs)	“omni,” “vl,” “vision,” audio models, “realtime” stacks	Capability varies by modality; “multimodal” can mean many different architectures [8]
Vision-language models (VLMs)	Image understanding + language	image+text → text	“vision,” “vl,” “vl-plus,” Llama 4, Gemini 3 Pro	“Vision” ≠ OCR quality; cropping/resize pipelines matter [9]

Category	What it does	Typical input → output	Where you see it	Common evaluation pitfalls
ASR (speech-to-text)	Transcribe speech	audio → text	Whisper-like models; cloud ASR offerings	Noise/accents dominate; "WER on test set" ≠ your mic setup [10]
TTS (text-to-speech)	Generate speech audio	text → audio	TTS APIs + open models	Latency vs expressiveness trade-off; voice cloning policies vary [11]
Image generation models	Generate/edit images	text/image → image	diffusion/ transformer diffusion families (SD 3.5, FLUX)	"Pretty sample" ≠ consistency/ editability; licenses vary [12]
Video generation models	Generate videos	text/image → video	Veo family; evolving vendor offerings	Availability and product status change quickly [13]
On-device / edge models	Run locally (CPU/GPU/NPU)	local text/ image/audio → local outputs	GGUF/MLX/ ONNX/TensorRT ecosystems	"File size" ≠ runtime RAM; KV cache dominates longer contexts [14]

What people confuse most: family vs checkpoint vs build

This distinction is the core "decoder ring."

Term	What it is	What changes	What does *not* change	Example of confusion
Model family	A branded lineage sharing training + design philosophy	Major training runs, tokenizer, architecture decisions	Not a single set of weights	"Claude" or "Llama" is not one model [15]
Checkpoint	A specific saved set of weights	Weights + sometimes config/tokenizer	The underlying architecture *type* (usually)	"Llama 4 Scout" checkpoint differs from "Maverick" [16]

Term	What it is	What changes	What does *not* change	Example of confusion
Parameter-size variant	Same recipe, different scale	Layer width/depth, parameter count	Brand/family intention	"1B vs 70B" are siblings, not "updates"
Fine-tuned derivative	Base weights adapted (SFT/RLHF/DPO/LoRA)	Behavior, refusal style, tool use, instruction-following	Most core knowledge and architecture, though behavior can shift a lot	"Instruct" isn't more knowledgeable; it's more aligned [17]
Distilled model	Student trained to imitate teacher outputs/trajectories	Often better "skill per parameter," sometimes narrower	It may inherit teacher biases; not necessarily same factual knowledge	"Distill" can outperform older bigger baselines on some tasks [18]
Quantized build	Same model, weights stored at lower precision	Accuracy/speed/VRAM trade-offs	The intended behavior (in theory), but small behavior changes happen	"7B Q4" is the *same* model family but a different artifact [19]
Serving/file format	How weights + metadata are packaged	Engine compatibility, load times, metadata inclusion	The conceptual model architecture	GGUF vs safetensors vs TensorRT engine are not "different models" [20]
Benchmark claim	A result under a specific harness	Prompting, tools, temperature, dataset versions	General real-world performance	"#1 on X" may reflect harness quirks or tool access [21]

Preview: how to read model names (real examples)

Below are "name decoding" previews; the full decoder comes later.

Real name	Quick decode	Operational implication	What it does *not* imply
`gpt-5.4-mini`	GPT family, smaller tier ("mini")	Lower cost/latency tier; may be used as subagent model [22]	Not open-weight; not necessarily best at deep reasoning
`claude-sonnet-4-6`	Claude family; Sonnet line; version 4.6	Use alias unless you need pinned versions; 1M context in beta is vendor-controlled [23]	"4.6" doesn't guarantee "better at everything"

Real name	Quick decode	Operational implication	What it does *not* imply
`gemini-3-pro`	Gemini 3 generation; Pro tier	Multimodal + long context; model string patterns include stable/preview/latest labels [24]	"Pro" is vendor-relative branding
`meta-llama/Llama-4-Scout-17B-16E-Instruct`	Llama 4; Scout; MoE with 16 experts; 17B *active*; instruct-tuned	MoE means "total params" differs from "active params"; open-weight under a custom license [25]	"16E" doesn't mean 16× compute; only some experts are active
`deepseek-ai/DeepSeek-R1-Distill-Qwen-14B`	DeepSeek R1 distilled into a Qwen 14B base	"Distill" signals teacher/student pipeline; common for "smarter small" reasoning models [26]	Not the original teacher; architecture and tokenizer follow Qwen base

Big comparison tables

This section is intentionally "high density." Use it as a fast reference.

The ecosystem in one page: function × artifact × runtime

You want to do…	Model type you likely need	Typical artifact	Common runtimes/serving	Biggest "gotcha"
General chat + writing + Q&A	General LLM (often "Instruct/Chat")	API model ID or open-weight Instruct	API; vLLM/TGI; llama.cpp/MLX for local [27]	"Base" models feel unhelpful unless you know prompting
Tool-using agents	Tool-use capable LLM	API ID; sometimes instruct fine-tunes	API tools; agent frameworks	Tool access changes benchmark outcomes [28]
Semantic search / RAG retrieval	Embeddings (+ often reranker)	API embedding endpoint; open embedding checkpoint	Vector DB + embedding service	Embeddings alone often underperform without reranking [29]
Sorting search results	Reranker (cross-encoder)	API rerank endpoint	Hosted rerank; sometimes local cross-encoder	Latency vs quality; needs candidate set first [7]

You want to do…	Model type you likely need	Typical artifact	Common runtimes/ serving	Biggest “gotcha”
OCR-like extraction from images/PDFs	VLM + OCR/ document models	VLM via API; specialized doc/ OCR model	Vendor doc stacks; custom pipelines	“Vision model” may describe images well but miss tiny text
Transcription	ASR	Whisper-like checkpoint or cloud model	Local GPU/CPU; cloud ASR	Real-time constraints make “best WER” less relevant [30]
Voice agent	Low-latency audio model	Realtime speech-to-speech model ID	Realtime APIs	Context + latency constraints differ from text chat [31]
Image generation/ editing	Image generator (diffusion/DiT-like)	Diffusers checkpoint; vendor API	Diffusers; vendor endpoints	Licenses and safety filters vary significantly [32]
Video generation	Video generator	Vendor API	Cloud vendor APIs	Product availability is volatile (deprecations) [33]
Local offline assistant	SLM/LLM + quantized build (GGUF/AWQ/etc)	GGUF or GPU quant	llama.cpp / MLX / vLLM	KV cache can dominate memory for long contexts [34]

“What matters vs noise” (the brutally practical table)

The point of this table is to stop you from over-weighting marketing tokens.

Label / feature	Usually important?	Why it matters (or not)	Common beginner mistake
Parameter count (e.g., 7B vs 70B)	Often	Correlates with capacity, but is not destiny; training and tuning dominate within a size band [35]	Assuming “bigger = always better”
MoE total params	Sometimes	Total parameters describe capacity; **active parameters** describe per-token compute [36]	Comparing MoE total params to dense params directly
Active params (MoE)	Often	Better proxy for latency/compute than total params [37]	Ignoring that MoE can still be heavy on memory/bandwidth

Label / feature	Usually important?	Why it matters (or not)	Common beginner mistake
Base vs Instruct	Extremely	Base = raw completion model; Instruct = trained to follow commands and chat norms [38]	Using base expecting ChatGPT-style behavior
“Chat” vs “Instruct” naming	Moderately	Often synonymous; vendor/community conventions differ	Treating them as strictly defined technical categories
Context length claim (32k/128k/1M/10M)	Often	Changes what you can feed at once; also changes cost and KV cache memory [39]	Assuming you can *afford* max context in production
Benchmark score	Sometimes	Useful when independent and replicable; fragile otherwise [21]	Believing vendor single-number claims
“Reasoning” branding	Sometimes	Can indicate training + inference-time compute controls (effort/thinking) [40]	Confusing “longer answers” with “better reasoning”
“Preview/experimental/latest”	Often	Indicates versioning stability and deprecation risk [41]	Building production systems on unstable model strings
“Mini/flash/turbo”	Sometimes	Usually a latency/cost tier with vendor-specific meaning [42]	Treating these suffixes as universal standards
Quantization method (AWQ/GPTQ/GGUF/EXL2)	Often	Determines hardware compatibility + quality/speed at same bit width [43]	Thinking “4-bit is 4-bit” regardless of method
Quantization level (Q4/Q5/Q8 etc)	Often	Affects accuracy and VRAM; Q8 is usually safer; Q4 often “good enough” for chat [44]	Picking lowest bits then blaming the base model
File format (GGUF/safetensors/ONNX)	Important for deployment	Controls which engines can load it and what metadata is embedded [45]	Confusing “format” with “method”
Tokenizer differences	Sometimes	Token boundaries affect cost and context, and can break “drop-in” swaps	Assuming all tokenizers behave similarly
License	Extremely	Determines legal use, redistribution, fine-tuning and commercial constraints [46]	Assuming “open-weight = open-source”

Functional taxonomy

This taxonomy is **by job**, not vendor. Representative current families are included where they are materially relevant today.

General-purpose LLMs

What it is. Broad language + instruction following, often the default "assistant brain."
Used for. Q&A, drafting, summarization, customer support, analysis, workflow automation.
Trade-offs. Quality vs cost vs latency; tool access and safety behavior differ; long-context increases memory/cost. [47]
Beginner misunderstanding. Treating different *tiers* as "the same model, just faster."

Representative current families.
Closed: GPT-5.4 line (with mini/nano tiers) [48]; Claude 4.6 line (Opus/Sonnet/Haiku tiers) [49]; Gemini 3 Pro/Flash family [24].
Open-weight: Llama 4 Scout/Maverick (MoE, multimodal) [50]; Mistral Large 3 (open-weight multimodal) [51]; Qwen 3.5 Omni (omni-modal line) [52].

Small local / edge LLMs

What it is. Models designed to fit consumer GPUs, laptops, and sometimes mobile/NPUs — often via quantization and engine-specific formats.
Used for. Private/local assistants, offline workflows, on-device summarization, local code helpers.
Trade-offs. Smaller context and weaker robustness vs privacy/latency/cost control. Engines matter. [53]
Beginner misunderstanding. Equating "download size" with "it will fit in VRAM."

Representative families (open-weight oriented): Llama family (including Llama 4 and smaller earlier variants distributed for local use) [54]; Gemma family [55]; Phi family for small efficient reasoning variants [56].

Reasoning-oriented models

What it is. Models tuned and/or served with mechanisms that allocate more "thinking" (extra inference-time compute or reasoning tokens) to hard problems. [57]
Used for. Math, multi-hop planning, hard coding/debugging, structured decision tasks.
Trade-offs. Higher latency and cost; sometimes worse at "simple chat"; potential for confident but wrong long rationales. [58]
Beginner misunderstanding. "Reasoning" = "always better."

Representative families: OpenAI reasoning models with `reasoning.effort` controls [59]; Mistral Magistral (reasoning traces) [60]; Qwen "thinking mode" model variants (explicit "Thinking" columns in some listings) [61]; DeepSeek reasoning lines including distilled R1 derivatives [62].

Code-specialized models

What it is. Models trained for code completion (FIM), repo understanding, tool-using SWE agents, and debugging.

Used for. IDE completion, code review, test generation, multi-file refactors, agentic coding.
Trade-offs. Some code models are brittle at general chat; licensing and tool schemas differ.
Beginner misunderstanding. Treating "code completion" and "agentic SWE" as the same need.

Representative families: Codestral (code completion + FIM) [63]; Devstral 2 (agentic coding, multi-file) [64]; DeepSeek-Coder family incl. MoE variants and reported code-heavy pretraining mix for earlier DeepSeek Coder series [65]; Qwen Coder lines (e.g., "coder-plus/next") [66].

Long-context models

What it is. Models engineered to accept very long inputs (tens of thousands to millions of tokens).
Used for. Large document synthesis, codebase-level Q&A, long-horizon planning, multi-document RAG with fewer retrieval hops.
Trade-offs. Longer context increases KV cache memory and compute; "supports 1M" does not mean "cheap at 1M." [67]
Representative examples: Gemini 3 Pro (1M context) [68]; Llama 4 Scout (10M context marketed; model family is MoE) [69]; Claude 4.6 models with 1M context in beta (vendor controlled) [70].

Multimodal models and VLMs

What it is. Models that can read images (and sometimes audio/video) and respond in language (and sometimes produce audio).
Architectural reality check. "Multimodal" can mean: (a) a vision encoder + projector + LLM, or (b) more native fusion; vendors differ. (See architecture section.) [71]
Representative examples: Llama 4 models (native multimodal positioning; MoE) [50]; Gemini 3 Pro (text, audio, image, video, PDFs) [68]; OpenAI realtime/audio models (audio in/out) [72].

Embeddings and reranking (retrieval stack)

Embeddings. Vector representations for similarity search and clustering. OpenAI's embedding guide explicitly frames embeddings as numerical representations used for relatedness and retrieval use cases. [73]
Rerankers. Models that sort candidate passages by relevance; Cohere describes rerankers as sorting inputs by semantic relevance to a query, commonly used after first-stage retrieval. [74]
Representative APIs: OpenAI `text-embedding-3-*` [75]; Vertex AI embedding models including `gemini-embedding-001` being generally available per Google Developer Blog [76]; Cohere Rerank model family docs [74]; open embedding families like BGE-M3 [77].

Speech models: ASR and TTS

ASR: Whisper is described by its creators as a general-purpose speech recognition model trained on diverse audio and capable of multilingual recognition and translation. [78]
Cloud ASR: Google's Chirp 3 "Transcription" appears in Speech-to-Text release notes as GA and positioned as a multilingual ASR-specific generative model in API v2. [79]
TTS: ElevenLabs positions multiple TTS models with explicit latency tiers (Flash/Turbo/etc.) and provides developer docs for TTS capabilities. [11]

Image and video generation

Image: Stable Diffusion 3.5 Large is described in its model card as a “Multimodal Diffusion Transformer (MMDiT)” text-to-image model; Stability AI published SD 3.5 variants in an open release under its Community License. [80]
Image: FLUX.1 dev model card describes it as a “rectified flow transformer” and Black Forest Labs docs position FLUX as their model family. [81]
Video: Veo is explicitly documented in Vertex AI’s Veo video generation reference; Google also announced Veo 3.1 Lite on Vertex AI in early April 2026. [82]
Video volatility: OpenAI published deprecations and a help article with concrete shutdown dates for Sora experiences and API, demonstrating how quickly video products can change. [83]

Architectural taxonomy

The goal here is to let you infer real operational differences — **latency, memory, quantization friendliness, and serving complexity** — from architecture class.

Dense decoder-only transformers

What it is. The “GPT-like” family: autoregressive next-token prediction. Introduced by the Transformer architecture. [84]
Why it exists. Strong general text generation; straightforward scaling.
Strengths. High-quality generation; good for chat and code.
Weaknesses. Attention cost and KV cache grow with sequence length; long-context inference becomes memory-heavy. [85]
Serving implications. Continuous batching and KV cache management are major levers for throughput (vLLM is explicitly built around PagedAttention to reduce KV cache waste). [86]

Mixture-of-Experts transformers (MoE)

What it is. Sparse activation: router selects subsets (“experts”) per token. Switch Transformer frames MoE as a way to get huge parameter counts with roughly constant per-token compute via sparse activation. [87]
Why it exists. More “capacity” without full dense compute; better quality/compute trade-offs in some regimes.
What matters operationally.

- **Total params**: storage/capacity.
- **Active params**: per-token compute proxy.
  Mixtral’s paper describes selecting 2 experts per token and distinguishes total vs active parameters. [88]
  Examples.
- Mixtral 8x7B: paper describes 47B total with ~13B active per token. [88]

- Llama 4 Maverick: Meta describes 17B active and 400B total parameters (MoE) in the Llama 4 release. [50]
  Weaknesses. Routing overhead, memory/bandwidth complexity; can be harder to deploy efficiently.

Encoder-only transformers

What it is. BERT-style bidirectional encoders, typically for representations (classification, embeddings). [89]
Why it exists. For tasks where you want a fixed vector or token-level representations.
Where you see it. Many rerankers and embedding families are encoder-only or encoder-dominant.

Encoder–decoder (seq2seq) transformers

What it is. T5-style: encoder reads input, decoder generates output. [90]
Why it exists. Strong for translation/summarization; retriever-augmented generation historically used seq2seq components. The original RAG paper combines parametric seq2seq with retrieval over a Wikipedia index. [91]

Multimodal transformer compositions

In practice you'll see at least two common shapes:

Vision encoder + projector + LLM hybrid.
A dedicated vision backbone produces embeddings; a projector aligns them to the LLM token space. Many "VLM" products work like this even if not described in detail publicly.

More native multimodal fusion.
Meta's Llama 4 documentation emphasizes "natively multimodal" positioning and MoE structure; the Llama 4 model card materials and release language describe multimodal capability and MoE. [92]
(When internal fusion details are not fully public, treat "native" as a *capability label*, not a guaranteed architecture.)

Diffusion and transformer-diffusion image families

Diffusion basics. Diffusion probabilistic models generate by iteratively denoising; DDPM is a canonical reference. [93]
Transformer diffusion (DiT). DiT replaces U-Nets with transformer backbones; the DiT paper describes diffusion models with transformers. [94]
Modern image families. SD 3.5 Large is explicitly described as MMDiT; FLUX.1 dev is described as a rectified flow transformer. [95]

Speech architectures

Whisper is publicly released and described as a multitask speech recognition model; it is often discussed as an encoder-decoder Transformer approach for ASR/translation. [96]
Large vendor speech stacks (e.g., Chirp) are typically accessed as APIs with limited architectural disclosure; use official release notes and docs for confirmed claims. [97]

Emerging non-standard sequence models (state-space)

Mamba is explicitly presented as a selective state space model architecture aiming for linear-time sequence modeling. [98]
Practical relevance. These models matter conceptually for long-context efficiency, but most mainstream "model names" you encounter day-to-day in 2026 are still transformer-centered.

Retrieval-augmented stacks (deployment pattern, not one model)

RAG is best understood as **a stack**: embed → retrieve → rerank → generate, often with citations and caching. The original RAG formulation is a canonical reference for combining parametric generation with retrieval. [91]
In production, the "model name" may hide this: "the LLM" is only one component.

Naming and suffix decoding guide

This is the **decoder ring** section. The guiding rule:

> A model name is usually a **bag of hints** about (1) scale, (2) tuning, (3) modality/specialization, and (4) packaging — but vendors mix engineering signal with branding.

To meet your requirement on uncertainty: throughout this section, labels follow:

- **Confirmed fact**: directly supported by docs/papers.
- **Strong public evidence**: multiple reliable public sources or explicit vendor statements.
- **Informed inference**: deduction from common practice + partial disclosure.
- **Speculation**: explicitly uncertain; avoided unless necessary.

How to parse a model name: a practical token grammar

Most names can be understood as:

[Family]-[Generation]-[Specialization]-[Size]-[Tuning]-[Context]-[Quant/Format]-[Version/Snapshot]

Not all parts appear, and order differs by vendor/community.

Size / scale indicators (0.5B … 100B+)

What "B" means. Usually "billions of parameters," but MoE complicates it: total vs active. Switch Transformer explains sparse activation and why MoE can have huge total parameters with sparse per-token compute. [87]
MoE examples in the wild.

- Mixtral 8x7B paper: 47B total parameters, ~13B active per token. [88]
- Llama 4 Maverick: Meta explicitly cites 17B active vs 400B total. [99]

Why bigger isn't always better. InstructGPT showed that a **1.3B** fine-tuned model could be preferred to a **175B** base model in human evaluations — a canonical demonstration that *post-training and alignment* can dominate size in perceived helpfulness. [100]

Common suffixes and descriptors (base/instruct/reasoning/vl/etc.)

This table is the "what it usually means" guide — **not** a claim of universal standards.

Token you see	Usually means	Operational implication	What it does *not* imply
`base` / `pretrained`	Raw next-token model	Needs prompting; best for further fine-tuning	Not safe/harmless by default
`instruct` / `chat` / `it`	Instruction-tuned via SFT and often preference optimization	Works better as assistant	Not more knowledgeable than base
`sft`	Supervised fine-tuning stage	Improved instruction following	Not necessarily safe/aligned alone [100]
`rlhf` / `rl`	Preference tuning with human feedback (often RLHF-like)	Changes refusal/helpfulness style	Not a guarantee against hallucinations [100]
`dpo`	Direct Preference Optimization	Preference alignment without explicit RL loop	Not automatically "more factual" [101]
`reasoning` / `thinking`	Reasoning-tuned or inference-time scaling controls	Higher latency/cost; better on hard problems	Not always better on easy tasks [102]
`coder` / `code`	Code-specialized training	Better code completion/debugging	Not necessarily good at chit-chat [103]
`vl` / `vision` / `vlm`	Vision-language capability	Image+text inputs	Not equal to OCR/document parsing quality [104]
`omni`	Multiple modalities (often text+image+audio; vendor-specific)	Broader I/O options	Not a specific architecture standard [105]
`embed` / `embedding`	Embedding/vector model	Use for search, clustering, RAG	Not a chat model [73]

Token you see	Usually means	Operational implication	What it does *not* imply
`rerank` / `ranker`	Relevance scoring model	Used after retrieval	Not a first-stage retriever [7]
`distill` / `distilled`	Student trained from teacher outputs/ trajectories	Better "skill per parameter" possible	Not the same as "compressed weights" [106]
`teacher/student`	Explicit distillation framing	Expect imitation of teacher behavior	Not necessarily same knowledge cutoff
`32k/128k/1M/10M`	Context window tier	Changes input capacity + KV cache footprint	Not "free"; cost/ memory scale with length [39]
`tool-use` / `function calling`	Tool invocation trained/ supported	Better structured actions	Not guaranteed "agent reliability" [107]
`preview` / `experimental` / `latest`	Versioning stability tier	Deprecation risk; alias may shift	Not a quality guarantee [108]
`mini` / `nano` / `flash` / `turbo`	Vendor tiering (speed/cost)	Usually cheaper/ faster	Not comparable across vendors [42]

Quantization naming: method vs format vs runtime build

This is where most people get lost. Three different things get mashed together:

1) **Quantization method** (math + calibration + kernels): GPTQ, AWQ, etc.
2) **File/packaging format**: GGUF, safetensors, ONNX, TensorRT engines.
3) **Runtime-specific build**: e.g., a TensorRT-LLM engine compiled for a GPU; or a GGUF tuned for llama.cpp.

GGUF (format). GGUF is a binary file format designed by the llama.cpp project to store tensors and metadata in one file for fast loading and quantization workflows. [109]
llama.cpp quant types like `Q4_K_M`. llama.cpp documents quantization options and the existence of the "K" variants used in GGUF quantization. [110]
GPTQ / AWQ (methods). GPTQ is a post-training quantization approach for GPT-like transformers; AWQ is activation-aware weight quantization. [111]
EXL2 (format + method family). ExLlama is explicitly oriented around efficient local LLM inference with EXL2 quantization artifacts. [112]
bitsandbytes / QLoRA (training + runtime quantization tooling). QLoRA describes NF4 4-bit quantization plus training tricks; it's a training method, not just a file format. [113]

Quick cheat sheet: common tokens you'll see in local model names

Token	Category	Meaning in practice	Commonly paired with
`BF16` / `FP16` / `FP8`	precision	Weight precision tier	Vendor APIs; TensorRT; some open distributions [114]
`INT8` / `INT4`	precision	Quantized weights	ONNX/TensorRT; GPTQ/AWQ; GGUF [115]
`Q8_0` / `Q6_K` / `Q4_K_M`	llama.cpp quant type	GGUF quant variants (runtime-specific naming)	llama.cpp / GGUF ecosystems [110]
`GGUF`	file format	llama.cpp-oriented format embedding tokenizer+metadata	llama.cpp, Ollama (often), LM Studio
`AWQ`	method + kernels	Activation-aware weight quant (often 4-bit)	vLLM/TensorRT/TGI support matrices [116]
`GPTQ`	method	Post-training quantization for GPT-like models	vLLM/TGI/TensorRT; often 4-bit [117]
`QAT`	training recipe	Quantization-aware training (or finetuning)	Often appears in research and some model releases [118]
`PTQ`	recipe class	Post-training quantization	GPTQ/AWQ are PTQ-style families [119]
`ONNX`	format	Exported graph format; can be quantized	ONNX Runtime, edge deployment [120]
`TensorRT-LLM`	runtime/build	NVIDIA optimized serving/build system	GPU inference, FP8/INT8/4-bit options [121]
`vLLM`	runtime	High-throughput serving engine using PagedAttention	OpenAI-style serving; continuous batching [122]
`MLX`	runtime	Apple silicon-focused ML framework	MLX LM, mlx-lm packages [123]

Fully decoded example breakdowns (template-style)

These are "real-looking" and designed to teach pattern recognition. When tokens are runtime-specific (e.g., `Q4_K_M`), that is called out.

Example: `Model-X-7B-Base`

- `Model-X`: family label (could be vendor/community)
- `7B`: ~7 billion parameters (dense unless otherwise stated)

- `Base`: pretrained completion model
  Implies: best for fine-tuning; not necessarily chat-friendly.
  Does NOT imply: safety alignment, tool-use, or that it's comparable to another 7B trained differently.

Example: `Model-X-7B-Instruct`

- `Instruct`: instruction-tuned (often via SFT, sometimes preference optimization) [124]
  Implies: reads prompts like "do X" better; more assistant-like.
  Does NOT imply: superior raw knowledge vs base.

Example: `Model-X-4B-Distilled-Instruct`

- `Distilled`: trained to imitate a larger teacher (could be behavior or trajectory distillation) [125]
  Implies: may outperform older untuned 7B on reasoning-like tasks. [100]
  Does NOT imply: same tokenizer or same architecture as teacher.

Example: `Model-X-32B-MoE`

- `MoE`: mixture-of-experts; expect "total vs active" parameter split [126]
  Implies: deployment complexity; "32B" may be active or total depending on convention (often unclear).
  Does NOT imply: 32B compute per token.

Example: `Model-X-7B-GGUF-Q4_K_M`

- `GGUF`: file format for llama.cpp-like flows [127]
- `Q4_K_M`: llama.cpp quantization type naming [128]
  Implies: loads in llama.cpp-family engines; quality close to 4-bit with K-variant choice.
  Does NOT imply: "AWQ quality" or "GPTQ quality" — those are different quant pipelines.

Example: `Model-X-Coder-7B-Instruct-AWQ`

- `Coder`: code-specialized
- `AWQ`: activation-aware weight quantization method [129]
  Implies: likely intended for GPU kernels that support AWQ; faster than generic 4-bit in some stacks.
  Does NOT imply: GGUF compatibility (unless explicitly provided separately).

Lifecycle, distillation, and reasoning

This section explains how the thing you download (or call) is derived.

The canonical derivation chain (what changes at each stage)

A practical, modern lifecycle is:

Pretrained base → SFT / instruct → preference tuning (RLHF/DPO/etc.) → reasoning tuning / test-time compute hooks → domain specialization → distillation (optional) → quantization (optional) → format conversion / engine build → deployment

Key references for the "alignment" and "preference" stages:

- InstructGPT: SFT then RLHF-style fine-tuning with human feedback. [130]
- DPO: preference optimization framed as a simpler alternative to RLHF pipelines. [131]
- LoRA: parameter-efficient finetuning by injecting low-rank adapters into frozen weights. [132]
- QLoRA: finetuning with 4-bit quantized weights (NF4) and associated training methods. [113]

What "distilled" means in modern practice

Confirmed core idea. Distillation transfers behavior/knowledge from a larger teacher into a smaller student; the Hinton et al. paper is canonical for distillation principles. [133]
Modern variants you actually encounter in names:

- **Compression-style distillation:** make a smaller network approximate teacher outputs. [134]
- **Behavior distillation:** student learns "how the teacher answers," including style and refusal patterns (often via synthetic data). (Informed inference; patterns widely used in open releases.)
- **Chain-of-thought / trajectory distillation:** student trains on teacher reasoning traces or intermediate steps; recent work focuses on avoiding "overthinking" and improving efficiency. [135]
- **Synthetic data distillation:** teacher generates large datasets that become SFT corpora for the student (common in practice and explicitly referenced in some distill releases). [136]

Why a distilled 4B can beat an older plain 7B (mechanism). InstructGPT provides a concrete demonstration that tuning and feedback can outweigh raw size in preference/utility. [100] A distilled student may inherit higher-quality behaviors and problem-solving patterns than an older baseline trained without those signals.

What "reasoning model" means today (and common failure modes)

Confirmed vendor pattern. Some vendors explicitly expose *reasoning budgets* (e.g., `reasoning.effort`) and distinguish reasoning tokens from output tokens. [59]
Research framing. Test-time compute scaling is an active area; papers describe improving reasoning by spending more inference compute and verification. [137]

Key distinctions you should make:

- **Stronger base model**: better representation and generalization.
- **Step-by-step tuned model**: more inclined to structured reasoning.
- **More test-time compute**: sampling/verification/verifiers improve results. [138]
- **Hidden reasoning traces**: vendor may discard reasoning tokens and only expose summaries. [139]
- **Just longer answers**: verbosity without correctness improvement (common failure).

Common failure modes of "reasoning-branded" models (practical).

- Overthinking easy tasks → wasted tokens/latency (explicitly discussed as a risk in CoT distillation literature). [140]
- Confident wrong multi-step explanations (long rationales can be persuasive but incorrect; verification is non-trivial). [141]
- Tool misuse: "agentic" models can fail by calling tools incorrectly if tool schemas are weak (informed inference; tool schema quality is known to matter in practice).

Quantization, runtimes, and practical decoding checklist

This section is intentionally "operational." It explains why the same "7B" can behave differently depending on build and runtime.

Why quantization exists (and why it's not just "compression")

Quantization reduces precision (e.g., FP16 → INT4), shrinking weight memory and often improving speed — at some quality cost. GGUF is explicitly designed around quantization workflows for llama.cpp-family inference. [142]
For GPU serving, frameworks like vLLM and TensorRT-LLM support multiple quantization strategies and kernel paths. [143]

The quality vs VRAM trade-off (what "Q4 good enough" actually means)

A practical mental model:

- **Weights memory**: roughly proportional to parameters × bits-per-weight.
- **KV cache memory**: grows with context length, batch size/concurrency, and model dimensions; vLLM emphasizes KV cache efficiency with PagedAttention. [144]
- **Long context** makes KV cache a first-class constraint; vLLM explicitly supports FP8 KV cache quantization to reduce memory footprint. [145]
- FlashAttention reduces memory traffic for attention, enabling longer contexts and better efficiency in many transformer setups. [146]

Rule of thumb (informed inference, widely observed):

- Q4-class can be "good enough" for casual chat and some coding, but sensitive tasks (math proofs, tricky code edits, multilingual nuance) benefit from higher precision (Q6/Q8, FP16/BF16). The exact breakpoints depend on model family and kernels; method matters at equal bits. [147]

Why "7B GGUF Q4" is not the same as "7B AWQ"

Because these differ in **both** packaging **and** intended kernels:

- **GGUF Q4_K_M**: llama.cpp-oriented file format and quant type naming. [142]

- **AWQ**: activation-aware weight quantization method typically paired with GPU kernels; vLLM/TGI mention specific handling of GPTQ/AWQ models and kernels. [148]

Same base model weights can yield different output quirks after quantization (small distribution shifts, rounding effects) — especially for borderline prompts. (Informed inference.)

Runtime glossary (what each tool actually is)

- **llama.cpp**: C/C++ inference engine emphasizing broad hardware support and local inference; explicitly supports CPU+GPU hybrid inference and multiple backends. [149]
- **GGUF**: llama.cpp's file format storing tensors + metadata; designed for fast loading and quantization workflows. [127]
- **vLLM**: serving engine built around PagedAttention to reduce KV cache waste and improve throughput. [122]
- **TGI (Text Generation Inference)**: Hugging Face serving stack; launcher docs show quantization handling and sharding options. [150]
- **TensorRT-LLM**: NVIDIA inference/serving framework with multiple quantization and precision options. [121]
- **ONNX / ONNX Runtime**: portable graph format and runtime; supports quantized export paths. [120]
- **MLX / mlx-lm**: Apple silicon-focused ML framework; Transformers integration notes explain shared-memory behavior and safetensors support. [151]
- **Ollama**: packaging/UX layer; Modelfile is a blueprint to build/share models, and can build from GGUF or safetensors. [152]

A seven-step checklist for decoding any unfamiliar model name

Use this every time; it prevents most mistakes.

1) **Identify the family and issuer** (vendor or org).

- Is it a vendor API model ID (stable/preview/latest)? [153]
- Or an open hub path (e.g., `org/model` on Hugging Face)?

2) **Determine whether the name is an alias or a pinned version.**

- Anthropic explicitly notes that aliases can point to latest; use full names to pin. [154]
- Google Gemini docs describe version naming patterns (stable/preview/latest/experimental). [155]

3) **Extract the *function* tokens.**
`coder`, `embed`, `rerank`, `vl`, `omni`, `realtime`, `tts`, `asr`, etc. [156]

4) **Extract scale tokens.**
`7B`, `70B`, or MoE patterns like `8x7B`, `17B-16E`. If MoE, look for **active vs total**. [157]

5) **Extract tuning tokens.**
`base` vs `instruct/chat`, plus distill/reasoning markers. [158]

6) **Extract deployment tokens (quantization + format + runtime).**
`GGUF` + `Q4_K_M` means llama.cpp-land; `AWQ/GPTQ` means PTQ method families likely targeting GPU kernels; `ONNX/TensorRT` signals runtime builds. [159]

7) **Check the license and the hardware fit — including KV cache.**
Llama community licenses are explicitly not "free software" per FSF commentary; treat "open-weight" as "source-available unless proven otherwise." [160]
For long contexts and concurrency, account for KV cache scaling and engine features like PagedAttention / KV cache quantization. [161]

Worked examples: decoding real model names start-to-finish

Example one: `gpt-5.4-mini`

1) Family/issuer: GPT line by OpenAI [162] . [163]
2) Alias vs pinned: vendor model ID; OpenAI also uses dated snapshots/deprecations for some models (general platform pattern). [164]
3) Function: general LLM with tool use and multimodal reasoning positioning. [22]
4) Scale/tier: "mini" tier indicates cost/latency tiering. [22]
5) Tuning: marketed as capable for coding/tool use; not labeled "base," so you should treat as assistant-ready. [22]
6) Packaging: API-only (you don't choose GGUF/AWQ).
7) Fit: context window and pricing are given in the launch post (400k context for mini in that announcement). [22]
Not implied: Open weights, local deployability, or that "mini" equals another vendor's "mini."

Example two: `claude-sonnet-4-6`

1) Family/issuer: Claude family by Anthropic [165] . [166]
2) Alias vs pinned: Claude Code docs explicitly note aliases always point to latest; pin with full model name. [167]
3) Function: general LLM; Sonnet line optimized for broad tasks; supports tool calling. [168]
4) Scale: not encoded as "B"; Anthropic uses product-line naming rather than parameter counts.
5) Tuning: assistant-ready; "Sonnet" implies the "mid-tier" family line in their lineup. (Confirmed lineup approach via official overview.) [169]
6) Packaging: API; can also be accessed through partner platforms with explicit model IDs. [170]
7) Fit: context window features are described as "1M token context window in beta" for Sonnet 4.6 announcement. [171]
Not implied: That every request truly uses 1M tokens cheaply; beta features can change.

Example three: `gemini-3-pro`

1) Family/issuer: Gemini line by Google [172] (with Gemini models also associated with Google DeepMind [173]). [174]
2) Versioning: Gemini docs explain stable/preview/latest/experimental patterns. [155]
3) Function: multimodal reasoning model; Vertex AI docs describe multimodal inputs and 1M token context for Gemini 3 Pro. [68]

4) Scale: not encoded in name; tier is “Pro.”
5) Tuning: assistant-ready.
6) Packaging: API; can be used via Vertex AI endpoints. [68]
7) Fit: long context is a feature; costs and latency scale accordingly (KV cache implications apply in self-hosting contexts). [86]
Not implied: That “Pro” equals “Opus” or “GPT frontier” across vendors.

Example four: `meta-llama/Llama-4-Scout-17B-16E-Instruct`

1) Family/issuer: Llama by Meta [175]. [176]
2) Versioning: Llama provides model cards/prompt formats and versioning guidance. [177]
3) Function: general + multimodal (image input), text output. [178]
4) Scale: `17B` active parameters; `16E` = 16 experts; total params are larger (~109B total per public release descriptions). [179]
5) Tuning: `Instruct` means assistant-tuned.
6) Packaging: open-weight; you can find BF16 and quantized distributions in some releases (e.g., HF model card notes quantization options for Llama 4 releases). [180]
7) License: Llama 4 Community License Agreement is explicitly provided on llama.com; it is not necessarily OSI-open-source. [181]
Not implied: That it runs locally on consumer GPUs without heavy quantization; also “10M context” is a claim that must be considered with cost/memory realities. [182]

Example five: `unsloth/Llama-3.3-70B-Instruct-GGUF` **(plus** `…Q4_K_M.gguf`**)**

1) Family/issuer: Llama 3.3 family; instructions tuned. [183]
2) Artifact type: this is a **converted/packaged** GGUF artifact, not the canonical training checkpoint. [184]
3) Function: general chat LLM (instruct).
4) Scale: 70B dense model (text-only). [183]
5) Tuning: instruct (SFT/RLHF stated in the model info text on this distribution). [185]
6) Quantization tokens: `Q4_K_M` in file names match llama.cpp quant naming conventions. [186]
7) Runtime: intended for llama.cpp-family engines; will not “just work” in vLLM unless converted back to a supported checkpoint format. [187]
Not implied: That it matches BF16 performance; quant level matters.

Current model family survey (curated, “major families only”)

This is not exhaustive; it’s the set you’re most likely to encounter in 2026 product and open-weight ecosystems.

Family	Issuer	Open-weight?	Primary roles	Architectural notes (public)	Distinct naming conventions you'll see
GPT-5.4 line	OpenAI [162]	Closed	General LLM, reasoning/tiering (mini/nano), tool use	Reasoning tokens + `reasoning.effort` controls for reasoning models (public API docs) [188]	`gpt-5.4`, `gpt-5.4-mini`, `gpt-5.4-nano` [48]
Realtime/audio models	OpenAI	Closed	Voice agents, audio in/out	Realtime API supports speech-to-speech; model pages list context windows and modalities [189]	`gpt-realtime-*`, `gpt-audio-*` [190]
Claude 4.6 line	Anthropic [165]	Closed	General LLM, agents, coding	Model IDs and version pinning practices are explicitly documented [191]	`claude-opus-4-6`, `claude-sonnet-4-6`, `claude-haiku-4-5` [192]
Gemini 3	Google [172] / Google DeepMind [173]	Closed	Multimodal reasoning, long context	Vertex AI docs describe multimodal inputs and 1M context for Gemini 3 Pro [24]	`gemini-3-pro`, versioning labels (stable/preview/latest/experimental) [155]
Llama 4	Meta [175]	Open-weight (custom license)	General LLM + VLM	MoE with active vs total params; Llama site documents models and licensing [193]	`Llama-4-Scout-17B-16E`, … `Maverick-17B-128E` [194]
Mistral Large/Small + Mixtral	Mistral AI [195]	Mixed (some open, some premier)	General LLM, MoE, code, reasoning	Mixtral paper and docs describe MoE and active params; Mistral docs provide model lists and features [196]	Names like `mistral-*-latest`, `mixtral-8x7b`, `magistral-*` [197]
Qwen 3.x / 3.5	Alibaba Cloud [198]	Mixed (open + proprietary variants)	Chat, coder, omni, thinking mode	Public model listings show "thinking mode" and very large context windows for some variants [199]	`qwen3.5-*`, `qwen3-*-thinking`, `qwen3-coder-*` [200]

Family	Issuer	Open-weight?	Primary roles	Architectural notes (public)	Distinct naming conventions you'll see
DeepSeek R1 / Coder	DeepSeek [201]	Mixed (some open weights)	Reasoning + code	DeepSeek R1 repo describes distill variants; code family describes code-heavy training mix [202]	`DeepSeek-R1-Distill-*`, `deepseek-coder-*` [203]
Phi	Microsoft [204]	Mixed	Small efficient reasoning	Microsoft research posts describe Phi variants and capabilities [56]	`phi-*` tiers
Command / Rerank	Cohere [205]	Closed	Enterprise LLM + retrieval stack	Cohere docs describe rerank models and language support [206]	`rerank-v*`, embed lines
Grok	xAI [207]	Closed	General LLM + tools	Official xAI docs list model families and usage patterns [208]	`grok-*` variants
Nemotron	NVIDIA [209]	Mixed	Foundation + optimized deployments	NVIDIA pages describe Nemotron families and NIM distribution ecosystem [210]	Often appears via NIM model cards
SD 3.5	Stability AI [211]	Open-ish (community license)	Image generation	SD 3.5 model card describes MMDiT; Stability AI blog describes release variants [80]	`sd3.5-large`, `turbo`, etc.
FLUX	Black Forest Labs [212]	Mixed	Image generation/ editing	FLUX.1 dev described as rectified flow transformer; official docs position FLUX family [213]	`FLUX.1 [dev]`, `FLUX.2 *`

Glossary of common terms (short, precise)

Term	Meaning (practical)	Anchor reference
LLM	Large language model (general text generation)	Transformer lineage [84]
SLM	Small language model (practically: runs cheaper/local; not a strict cutoff)	InstructGPT size-vs-preference example [100]

Term	Meaning (practical)	Anchor reference
VLM	Vision-language model (image+text → text)	Gemini 3 Pro, Llama 4 docs [214]
MoE	Mixture-of-Experts; sparse activation	Switch Transformers, Mixtral, Llama 4 [36]
Dense	All parameters active per token	Standard transformer usage [84]
Checkpoint	Saved weights/config snapshot	Open model distribution practice [215]
Tokenizer	Text→token mapping; affects cost/context	(General concept; vendor-specific)
Embedding	Vector representation for similarity search	OpenAI embeddings guide [216]
Reranker / cross-encoder	Scores query-doc pairs for relevance	OpenSearch reranking tutorial explains cross-encoder role [217]
Decoder-only	Autoregressive generation architecture	Transformer paper lineage [84]
Encoder-only	Bidirectional encoding for representations	BERT [89]
Seq2seq	Encoder-decoder input→output mapping	T5 [218]
Context window	Max tokens processed at once	Gemini 3 Pro 1M; Llama 4 10M marketing [219]
KV cache	Stored keys/values for faster autoregressive decoding; memory grows with context/concurrency	vLLM paper + quantized KV cache docs [161]
SFT	Supervised fine-tuning	InstructGPT [100]
RLHF	Preference alignment via human feedback and RL loop	InstructGPT [100]
DPO	Preference optimization via classification objective	DPO paper [101]
Distillation	Teacher→student imitation	Hinton distillation paper [134]
QAT	Quantization-aware training/fine-tuning	Edge quantization research line [118]
PTQ	Post-training quantization	GPTQ/AWQ are PTQ-family methods [119]
Quantization	Lower-precision representation for weights/activations	GGUF + PTQ methods [220]
Hallucination	Confident-sounding incorrect output	Mitigated via retrieval/verification; not eliminated [221]

Term	Meaning (practical)	Anchor reference
Tool/function calling	Structured outputs to invoke tools/APIs	OpenAI reasoning/tooling docs; Mistral and Qwen tool-use docs [222]
Agent	System that uses model + tools + memory + planning	Vendor "agentic" positioning; not one model feature alone [28]
RAG	Retrieval-augmented generation stack	RAG paper [223]
LoRA	Low-rank adapters for efficient fine-tuning	LoRA paper [224]
QLoRA	LoRA applied with 4-bit quantization + training tricks	QLoRA paper [225]
Speculative decoding	Draft+verify acceleration technique	Recent speculative decoding work + vLLM context [226]
Test-time compute	Spending more inference compute to improve answers	Test-time scaling literature [227]

Key takeaways and rules of thumb

1) **Always separate: function → architecture → tuning → packaging.** Most "model name confusion" is those layers getting mixed.
2) **Instruct beats base for most users.** If you're not training, start from instruct/chat variants. [100]
3) **MoE requires two numbers in your head:** total params and active params. Compare accordingly. [37]
4) **Quantization is not one thing.** "INT4" is a precision; GPTQ/AWQ/GGUF are pipelines; TensorRT/vLLM/llama.cpp are runtimes. [228]
5) **Context length is a budget, not a trophy.** KV cache and throughput constraints dominate real deployments. [229]
6) **Treat vendor "mini/flash/turbo" as branding until proven otherwise.** Use docs for actual modality/context/limits. [230]
7) **Prefer pinned versions for production.** Alias strings move; deprecations happen. [231]

---

[1] [2] [27] [47] [190] https://developers.openai.com/api/docs/models
https://developers.openai.com/api/docs/models

[3] [22] [28] [42] [48] [107] [163] https://openai.com/index/introducing-gpt-5-4-mini-and-nano/
https://openai.com/index/introducing-gpt-5-4-mini-and-nano/

[4] [40] [57] [58] [59] [102] [139] [188] [222] https://developers.openai.com/api/docs/guides/reasoning/
https://developers.openai.com/api/docs/guides/reasoning/

[5] [63] [156] [201] https://docs.mistral.ai/models/codestral-25-08
https://docs.mistral.ai/models/codestral-25-08

[6] [29] [73] [216] https://developers.openai.com/api/docs/guides/embeddings/
https://developers.openai.com/api/docs/guides/embeddings/

7 74 206 https://docs.cohere.com/docs/rerank
https://docs.cohere.com/docs/rerank

8 72 https://developers.openai.com/api/docs/models/gpt-realtime-1.5
https://developers.openai.com/api/docs/models/gpt-realtime-1.5

9 16 71 92 104 177 178 https://www.llama.com/docs/model-cards-and-prompt-formats/llama4/
https://www.llama.com/docs/model-cards-and-prompt-formats/llama4/

10 https://openai.com/index/whisper/
https://openai.com/index/whisper/

11 https://elevenlabs.io/text-to-speech-api
https://elevenlabs.io/text-to-speech-api

12 80 95 212 https://huggingface.co/stabilityai/stable-diffusion-3.5-large
https://huggingface.co/stabilityai/stable-diffusion-3.5-large

13 33 82 https://docs.cloud.google.com/vertex-ai/generative-ai/docs/model-reference/veo-video-generation
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/model-reference/veo-video-generation

14 53 149 https://github.com/ggml-org/llama.cpp
https://github.com/ggml-org/llama.cpp

15 166 169 https://platform.claude.com/docs/en/about-claude/models/overview
https://platform.claude.com/docs/en/about-claude/models/overview

17 35 38 100 124 130 158 172 https://arxiv.org/abs/2203.02155
https://arxiv.org/abs/2203.02155

18 106 125 133 134 https://arxiv.org/abs/1503.02531
https://arxiv.org/abs/1503.02531

19 44 110 128 147 173 186 Quantizing Models - llama.cpp
https://mintlify.com/ggml-org/llama.cpp/models/quantizing-models?utm_source=chatgpt.com

20 34 45 109 127 142 159 184 187 204 220 228 GGUF File Format - llama.cpp
https://mintlify.com/ggml-org/llama.cpp/concepts/gguf-format?utm_source=chatgpt.com

21 https://docs.mistral.ai/getting-started/models/compare?models=magistral-medium-1-2-25-09
https://docs.mistral.ai/getting-started/models/compare?models=magistral-medium-1-2-25-09

23 171 https://www.anthropic.com/news/claude-sonnet-4-6
https://www.anthropic.com/news/claude-sonnet-4-6

24 39 68 174 214 219 https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/3-pro
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/3-pro

25 176 194 https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E
https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E

26 https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-14B
https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-14B

30 78 96 https://github.com/openai/whisper
https://github.com/openai/whisper

31 189 https://developers.openai.com/api/docs/guides/realtime/
https://developers.openai.com/api/docs/guides/realtime/

32 https://stability.ai/news-updates/introducing-stable-diffusion-3-5
https://stability.ai/news-updates/introducing-stable-diffusion-3-5

36 87 126 https://arxiv.org/abs/2101.03961
https://arxiv.org/abs/2101.03961

37 88 157 196 https://arxiv.org/abs/2401.04088
https://arxiv.org/abs/2401.04088

41 154 162 167 231 https://code.claude.com/docs/en/model-config
https://code.claude.com/docs/en/model-config

43 https://github.com/turboderp-org/exllamav2
https://github.com/turboderp-org/exllamav2

46 181 https://www.llama.com/llama4/license/
https://www.llama.com/llama4/license/

49 70 https://www.anthropic.com/news/claude-opus-4-6
https://www.anthropic.com/news/claude-opus-4-6

50 99 179 193 https://ai.meta.com/blog/llama-4-multimodal-intelligence/
https://ai.meta.com/blog/llama-4-multimodal-intelligence/

51 https://docs.mistral.ai/getting-started/models
https://docs.mistral.ai/getting-started/models

52 105 https://qwen.ai/blog?id=qwen3.5-omni
https://qwen.ai/blog?id=qwen3.5-omni

54 215 https://www.llama.com/docs/getting_the_models/
https://www.llama.com/docs/getting_the_models/

55 Gemma releases | Google AI for Developers
https://ai.google.dev/gemma/docs/releases?utm_source=chatgpt.com

56 Phi-4-reasoning-vision and the lessons of training a ...
https://www.microsoft.com/en-us/research/blog/phi-4-reasoning-vision-and-the-lessons-of-training-a-multimodal-reasoning-model/?utm_source=chatgpt.com

60 197 https://docs.mistral.ai/capabilities/reasoning
https://docs.mistral.ai/capabilities/reasoning

61 199 200 209 https://www.alibabacloud.com/help/en/model-studio/models
https://www.alibabacloud.com/help/en/model-studio/models

62 136 202 https://github.com/deepseek-ai/deepseek-r1
https://github.com/deepseek-ai/deepseek-r1

64 https://docs.mistral.ai/models/devstral-2-25-12
https://docs.mistral.ai/models/devstral-2-25-12

65 103 https://github.com/deepseek-ai/DeepSeek-Coder
https://github.com/deepseek-ai/DeepSeek-Coder

66 https://www.alibabacloud.com/help/en/model-studio/qwen-coder
https://www.alibabacloud.com/help/en/model-studio/qwen-coder

67 85 86 144 161 229 https://arxiv.org/pdf/2309.06180
https://arxiv.org/pdf/2309.06180

69 182 https://www.llama.com/models/llama-4/
https://www.llama.com/models/llama-4/

75 https://developers.openai.com/api/docs/models/text-embedding-3-large
https://developers.openai.com/api/docs/models/text-embedding-3-large

76 198 https://developers.googleblog.com/gemini-embedding-available-gemini-api/
https://developers.googleblog.com/gemini-embedding-available-gemini-api/

77 https://bge-model.com/bge/bge_m3.html
https://bge-model.com/bge/bge_m3.html

79 97 https://docs.cloud.google.com/speech-to-text/docs/release-notes
https://docs.cloud.google.com/speech-to-text/docs/release-notes

81 213 https://huggingface.co/black-forest-labs/FLUX.1-dev
https://huggingface.co/black-forest-labs/FLUX.1-dev

83 https://help.openai.com/ru-ru/articles/20001152-what-to-know-about-the-sora-discontinuation
https://help.openai.com/ru-ru/articles/20001152-what-to-know-about-the-sora-discontinuation

84 https://arxiv.org/abs/1706.03762
https://arxiv.org/abs/1706.03762

89 https://arxiv.org/abs/1810.04805
https://arxiv.org/abs/1810.04805

90 175 218 https://arxiv.org/abs/1910.10683
https://arxiv.org/abs/1910.10683

91 221 223 https://arxiv.org/abs/2005.11401
https://arxiv.org/abs/2005.11401

93 https://arxiv.org/abs/2006.11239
https://arxiv.org/abs/2006.11239

94 https://arxiv.org/abs/2212.09748
https://arxiv.org/abs/2212.09748

98 https://arxiv.org/abs/2312.00752
https://arxiv.org/abs/2312.00752

101 131 https://arxiv.org/abs/2305.18290
https://arxiv.org/abs/2305.18290

108 153 155 230 https://ai.google.dev/gemini-api/docs/models
https://ai.google.dev/gemini-api/docs/models

111 117 119 Your CPU is NOT Broken: the hidden Truth about GGUF ...
https://medium.com/artificial-intel-ligence-playground/your-cpu-is-not-broken-the-hidden-truth-about-gguf-quantization-d41a70287a1e?utm_source=chatgpt.com

112 https://github.com/Minami-su/vllm-gptq
https://github.com/Minami-su/vllm-gptq

113 225 https://arxiv.org/abs/2305.14314
https://arxiv.org/abs/2305.14314

114 https://docs.oracle.com/en-us/iaas/Content/generative-ai/meta-llama-3-3-70b.htm
https://docs.oracle.com/en-us/iaas/Content/generative-ai/meta-llama-3-3-70b.htm

115 205 https://mintlify.com/NVIDIA/TensorRT-LLM/features/quantization
https://mintlify.com/NVIDIA/TensorRT-LLM/features/quantization

116 143 https://arxiv.org/pdf/2312.03788
https://arxiv.org/pdf/2312.03788

118 https://arxiv.org/abs/2505.18724
https://arxiv.org/abs/2505.18724

120 211 https://huggingface.co/transformers/v4.4.2/serialization.html
https://huggingface.co/transformers/v4.4.2/serialization.html

121 https://github.com/NVIDIA/TensorRT-LLM/blob/main/examples/quantization/README.md
https://github.com/NVIDIA/TensorRT-LLM/blob/main/examples/quantization/README.md

122 https://arxiv.org/abs/2309.06180
https://arxiv.org/abs/2309.06180

123 https://ml-explore.github.io/mlx/build/html/examples/llama-inference.html
https://ml-explore.github.io/mlx/build/html/examples/llama-inference.html

129 What is GGUF? Complete Guide to GGUF Format
https://ggufloader.github.io/what-is-gguf.html?utm_source=chatgpt.com

132 224 https://arxiv.org/abs/2106.09685
https://arxiv.org/abs/2106.09685

135 140 https://arxiv.org/abs/2602.21786
https://arxiv.org/abs/2602.21786

137 227 https://aclanthology.org/2025.emnlp-main.1025.pdf
https://aclanthology.org/2025.emnlp-main.1025.pdf

138 https://arxiv.org/pdf/2502.20379
https://arxiv.org/pdf/2502.20379

141 165 https://openreview.net/forum?id=wl3eI4wiE5
https://openreview.net/forum?id=wl3eI4wiE5

145 https://docs.vllm.ai/en/latest/features/quantization/quantized_kvcache/
https://docs.vllm.ai/en/latest/features/quantization/quantized_kvcache/

146 https://arxiv.org/abs/2205.14135
https://arxiv.org/abs/2205.14135

148 150 https://huggingface.co/docs/text-generation-inference/reference/launcher
https://huggingface.co/docs/text-generation-inference/reference/launcher

151 https://huggingface.co/docs/transformers/community_integrations/mlx
https://huggingface.co/docs/transformers/community_integrations/mlx

152 https://docs.ollama.com/modelfile
https://docs.ollama.com/modelfile

160 https://www.fsf.org/blogs/licensing/llama-3-1-community-license-is-not-a-free-software-license
https://www.fsf.org/blogs/licensing/llama-3-1-community-license-is-not-a-free-software-license

164 https://developers.openai.com/api/docs/deprecations/
https://developers.openai.com/api/docs/deprecations/

168 170 192 https://docs.cloud.google.com/vertex-ai/generative-ai/docs/partner-models/claude/use-claude
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/partner-models/claude/use-claude

180 https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E-Instruct
https://huggingface.co/meta-llama/Llama-4-Scout-17B-16E-Instruct

183 195 https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct
https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct

185 207 https://huggingface.co/unsloth/Llama-3.3-70B-Instruct-GGUF
https://huggingface.co/unsloth/Llama-3.3-70B-Instruct-GGUF

191 https://platform.claude.com/docs/en/api/models/list
https://platform.claude.com/docs/en/api/models/list

203 https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-8B
https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-8B

208 Models and Pricing
https://docs.x.ai/developers/models?utm_source=chatgpt.com

210 NVIDIA Nemotron
https://www.nvidia.com/en-us/ai-data-science/foundation-models/nemotron/?utm_source=chatgpt.com

217 https://docs.opensearch.org/latest/tutorials/reranking/reranking-cohere/
https://docs.opensearch.org/latest/tutorials/reranking/reranking-cohere/

226 https://arxiv.org/pdf/2603.03251
https://arxiv.org/pdf/2603.03251